RAG-Based Customer Support Assistant for E-commerce

1. Problem Statement

E-commerce platforms receive a large number of customer queries related to order tracking, refunds, cancellations, and returns. Handling these queries manually through customer support teams leads to delays, increased operational costs, and inconsistent responses.

Customers often expect instant and accurate answers, but human agents may not always be available, especially during peak hours. This creates a need for an automated system that can efficiently handle common queries while maintaining accuracy and reliability.

2. Proposed Solution

To address this problem, we propose a **Retrieval-Augmented Generation (RAG) based Customer Support Assistant**.

The system will:

- Use a PDF-based knowledge base containing customer support policies and FAQs
- Convert the document into chunks and generate embeddings
- Store embeddings in a vector database (ChromaDB)
- Retrieve relevant information based on user queries
- Generate contextual responses using a Large Language Model (LLM)
- Use a graph-based workflow (LangGraph) to control execution flow
- Route queries based on intent and confidence
- Escalate complex or unclear queries to a human agent using a Human-in-the-Loop (HITL) mechanism

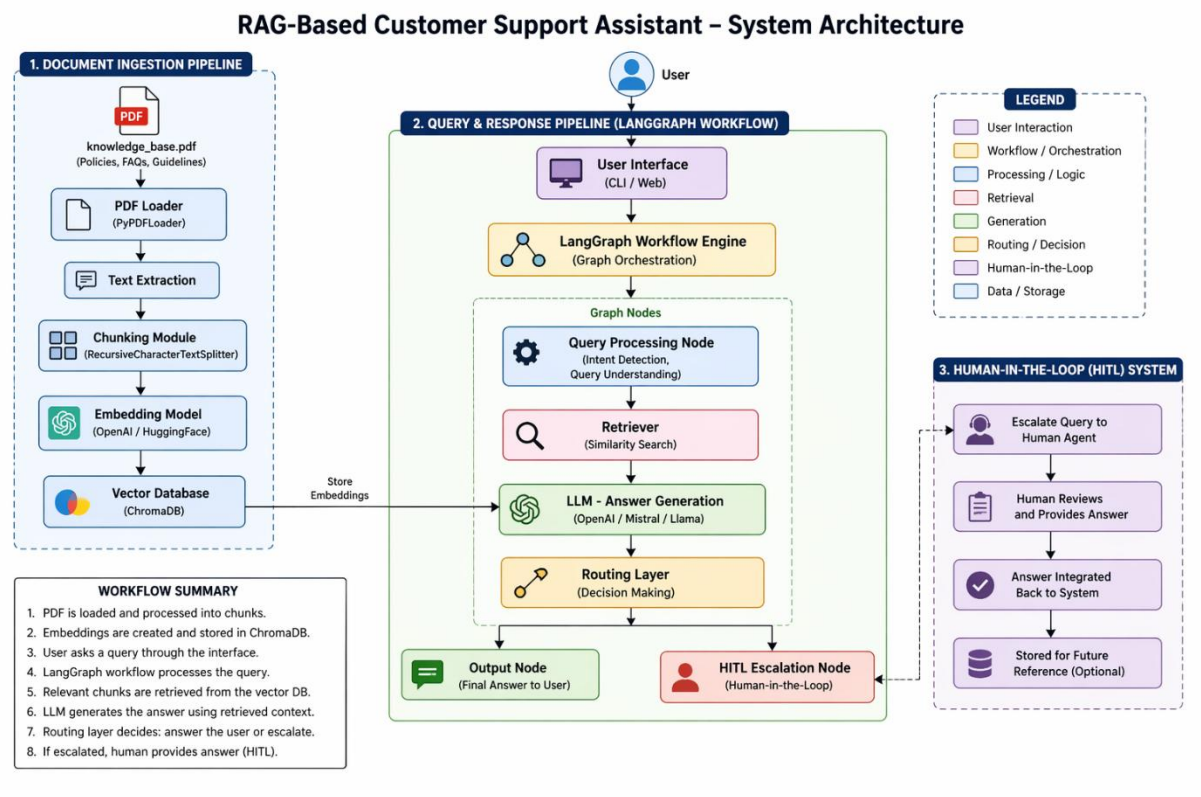
3. Scope of the System

The system is designed to:

- Answer frequently asked customer queries
- Provide accurate responses using document-based retrieval
- Reduce the workload on human support agents
- Improve response time and customer experience
- Identify complex queries and escalate them to human agents

The system does not handle transactional operations such as placing orders or processing payments. It focuses only on providing information and support.

System Architecture



6. System Architecture Explanation

The system is designed using a Retrieval-Augmented Generation (RAG) architecture combined with a graph-based workflow using Lang Graph.

The architecture consists of two main pipelines:

1. Document Processing Pipeline

The knowledge base in PDF format is first loaded using a PDF loader. The text is then extracted and divided into smaller chunks using a chunking strategy. These chunks are converted into vector embeddings using an embedding model. The embeddings are stored in a vector database (ChromaDB) for efficient similarity-based retrieval.

2. Query Processing Pipeline

When a user submits a query through the user interface, it is passed to the LangGraph workflow engine. The query is processed in the query processing node, where intent detection and understanding occur.

The retriever then fetches the most relevant chunks from the vector database. These retrieved chunks are passed to the Large Language Model (LLM), which generates a contextual response.

3. Routing and Decision Making

After the response is generated, a routing layer evaluates the quality and confidence of the answer. Based on predefined conditions, the system either:

- Sends the response back to the user
- Escalates the query to a human agent (HITL)

4. Human-in-the-Loop (HITL)

If the system is unable to provide a confident or accurate response, the query is forwarded to a human agent. The human reviews the query and provides the correct response, which is then returned to the user.

7. Component Description

The system consists of the following key components:

- **PDF Loader:** Responsible for loading the knowledge base PDF and extracting text content from it.
- **Chunking Module:** Splits the extracted text into smaller, manageable chunks to improve retrieval accuracy.
- **Embedding Model:** Converts each text chunk into a numerical vector representation for similarity-based search.
- **Vector Database (ChromaDB):** Stores the embeddings of all document chunks and enables efficient retrieval.
- **Retriever:** Searches the vector database and fetches the most relevant chunks based on the user query.
- **Large Language Model (LLM):** Generates context-aware and natural language responses using the retrieved information.
- **LangGraph Workflow Engine:** Manages the flow of execution between different components using a graph-based structure.
- **Routing Layer:** Evaluates the generated response and decides whether to return it or escalate the query.
- **Human-in-the-Loop (HITL) Module:** Handles escalation of complex queries to a human agent for accurate resolution.

8. Data Flow

The data flow in the system follows a structured pipeline from document ingestion to response generation.

First, the knowledge base PDF is loaded into the system using a document loader. The extracted text is then divided into smaller chunks using a chunking strategy.

Next, each chunk is converted into vector embeddings using an embedding model. These embeddings are stored in a vector database (ChromaDB) for efficient similarity-based retrieval.

When a user submits a query, it is processed by the LangGraph workflow engine. The system then uses a retriever to search the vector database and fetch the most relevant chunks.

These retrieved chunks are passed to the Large Language Model (LLM), which generates a contextual response based on both the query and the retrieved information.

Finally, a routing layer evaluates the response. If the response is accurate and confident, it is returned to the user. Otherwise, the query is escalated to a human agent through the Human-in-the-Loop (HITL) mechanism.

9. Technology Choices

The system uses the following technologies:

- **ChromaDB:** Used as the vector database due to its lightweight nature and ease of integration for storing embeddings.
- **LangGraph:** Used to implement a graph-based workflow for managing control flow and decision-making.
- **Embedding Model:** Used to convert text into vector representations for similarity-based retrieval.
- **Large Language Model (LLM):** Used for generating natural language responses.
- **Python:** Used as the primary programming language due to its strong ecosystem for AI and machine learning.

These technologies are selected to ensure efficient performance, flexibility, and ease of development.

10. Scalability Considerations

To ensure scalability, the system is designed to handle increasing data and user load efficiently.

- The vector database can be replaced with scalable solutions such as FAISS or Pinecone when handling large datasets.
- Chunking strategies can be optimized to balance context quality and retrieval speed.
- Caching mechanisms can be implemented to reduce repeated computations and improve response time.
- The system can support increased query load using asynchronous processing techniques.
- Load balancing can be applied to distribute incoming requests efficiently across the system.

These considerations help ensure that the system performs reliably even as the number of users and documents increases.